Roll Number: _____ **Date:** December 16, 2025

Thapar Institute of Engineering & Technology, Patiala

Department of Computer Science and Engineering

END Semester Test (EST)

B.E. (III-rd Semester) **Time**: 09.00 am-12.00 noon

Course Name (Code): Object Oriented Programming (UTA018) **Duration**: 3 Hours **M. Marks**: 40

Instructor(s) / Groups : Raman: (2C11-18, UOQ, NSW, TCD), Suresh: (2C21-25, 2C61-65), Kapil : (2C31-35), Subhrendu: (2C41-45, 2Q31-35), Vaishali : (2C51-55, 2Q21-25), Harpreet: (2C71-75, 2C81-82, 2Q41), Himanshu: (2Q11-15), Tanu : (2P11-14)

Instructions:

(i) Attempt ANY three questions. Each question is of 13 Marks. One mark will be awarded for attempting the questions in the order.

(ii) Consider all given programs are syntactically correct. Assume missing data, if any, suitably.

Q1 — Marks: 02 | CO/BL: 03/03

A. Explain the main advantage of virtual destructors in not more than three sentences.

Marks: 07

B. Write a program in C++ to demonstrate different types of constructors using constructor overloading.

Marks: 04

C. Fill in each <***BLANK***> to allocate memory to **as a 2D array.

```
#include<iostream>
using namespace std;
 class Matrix {   int **a;
public:
   Matrix(int r=0, int c=0) {
      <BLANK>
        for (int i = 0; i < r; i++)
            <BLANK>}
};
int main() {
   Matrix m(5, 4 );
}
```

Q2 — Marks: 05 | CO/BL: 04/03

A. Complete and correct the code below by filling each **<*BLANK*>** with appropriate C++ code. Predict the output of the code also.

```
#include <iostream>
#include <list>
using namespace std;

class MyList:public std::list <BLANK>{
   int x=2;
public:
<BLANK> multiplyBy2() {
      return x * 2; }

int operator()(int (MyList::*pmf)()) {
      return (<BLANK>)();
   } };
```

```
int main() {
MyList L;
L.push_back(1);   L.push_back(2);
L.push_back(3);

int (MyList::*ptr)() = &MyList::multiplyBy2;

cout << "Calling the Function:";
cout << L(ptr) << endl;

return 0;
}
```

Marks: 08

B. Consider the following C++ code snippet. The template class PTR_Arr<T> implements a container that stores pointers to dynamically allocated objects of **type T**. It provides indexed access and allows deletion of objects whose specific data member matches a given value, using a pointer-to-member. All deleted objects are properly freed to prevent memory leaks. Fill in the sections marked **<*BLANK*>** using appropriate C++ concepts and language constructs only.

```
template<typename T>
class PTR_Arr {
   T** p;
   size_t n;
   const size_t CAP;
public:
   PTR_Arr (): p(nullptr), n(0), CAP(2){
      p = new <BLANK>; }

   ~ PTR_Arr () {
      for (size_t i = 0; i < n; i++)
            delete p[i];
      <BLANK> ; }
```

```
    T* operator[](size_t i) const { return p[i]; }
   size_t size() const { return n; }
   <BLANK>;
   size_t remove (M T::*mp, const M& val) {
      size_t w = 0, removed = 0;
       for (size_t i = 0; i < n; i++) {
         T* obj = p[i];
         if(obj->*mp ==val){ <BLANK>;
                              removed++;}
         else p[w++] = obj;
       } n = w;
      return removed;
} };
```

Q3 **A.** Write a C++ program to provide the solution for ambiguity in hybrid (diamond) inheritance. — 07 — 03/03

B. In the C++ program given below, ***Line 22*** should be replaced with a function call inside main(). You must replace ***Line 22*** with each of the statements listed in Part (i) and Part (ii) and, for each case. Write the exact output produced, in the correct sequence and Explain the output(s) briefly. Clearly refer to line numbers from the program in your answer. — 06 — 04/03

(i) risky(-0.0); **(ii)** risky(-0.01);

```
1. #include <iostream>
2. #include <exception>
3. using namespace std;
4. class A {
5. public:
6. A()  { cout << "A()\n"; }
7. ~A() { cout << "~A()\n"; }
8. };

9. class MyError{
10. public:
11.       const char* what() const {return
"MyError occurred";}
12. };
```

```
13. void risky(bool t) {
14. A obj;
15. try { t ? throw MyError():throw 5;}
16. catch (int) {
17. cout << "Caught int in risky()\n";
18. throw; }
19. }
20. int main() {
21. try {
22. // Replace Line here without
comments
23. }catch (MyError e) {  cout << e.what() <<
"\n"; }
24. catch (...) { cout << "Unknown\n";}}
```

Write a C++ program to **define** a class ***FLOAT*** with: — 03/03

A. Two private integer variables: ***mantissa*** and ***exponent*** and a parameterized constructor with default parameters to initialize both members. Write appropriate getter and setter methods. — 02

B. Overload the following operators as member functions — 04
- i. Arithmetic operators: + and * (to achieve floating point arithmetic)
- ii. Comparison operator: ==
- iii. Assignment operator: =

C. Overload arithmetic operators (+, *) to work between a ***FLOAT*** object and an integer (int) using a friend function. — 03

D. Write a main() function to demonstrate: — 04
- i. All the arithmetic operations between two ***FLOAT*** objects
- ii. Comparison using ==
- iii. Assignment using =
- iv. Arithmetic between a ***FLOAT*** object and an *int*

Note: (i) You must perform all arithmetic and comparison operations directly on the mantissa and exponent integers stored inside the class ***FLOAT***.

(ii) You must ***NOT*** convert the ***FLOAT*** numbers into C++ built-in floating-point types (like `float` or `double`) within any of your operator implementations.

(iii) A ***FLOAT*** number N is internally represented as **N=mantissa×10^{exponent}**. For example, −0.56 is represented as mantissa=−56 and exponent=−2, while 10.234 is represented as mantissa=10234 and exponent=−3. Assume the addition of these numbers will be a ***FLOAT* X,** then:

exponent(X) = min (exponent(-.56) , exponent(10.234))

mantissa (X) = mantissa(-.56) *10$^{|\text{exponent (-.56) - exponent (10.234)}|}$ + mantissa (10.234)

where, |D| represents the modulus of D. *You need to work out the rest of the operator logic yourself based on this understanding.*